wp-15-vortex-addressed-semantic-memory

Vortex-Addressed Semantic Memory: Retrieval by Archetypal Geometry Rather Than Embedding Similarity

Authors: Weslyn Whitehead Jr.[1]

Affiliations:
[1] AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: https://orcid.org/0009-0005-7707-3210

Submitted: May 2026 · **Revision 2:** June 2026
Working Paper Series: AAMT-WP-15
Anchor DOI: 10.5281/zenodo.19600795

> **Revision 2 changes:** (C8) §2.2 replaces the raw L2 metric with the closed-form **product Fisher–Rao** distance (the natural information geometry of the product-Bernoulli Meji measure); §8 marks the former future-work item resolved and notes the conserved-manifold restriction (WP-01 §2.1a). No other content changed.

Abstract

Semantic memory retrieval in language model systems is almost universally implemented as nearest-neighbor search over dense vector embeddings: a query is encoded by a sentence transformer, and stored memories are ranked by cosine similarity to the query vector. We present **Vortex-Addressed Semantic Memory (VASM)**, a complementary retrieval paradigm in which memories are stored and retrieved by their TERA coordinate — a four-dimensional vector in the unit hypercube derived from the symbolic Vortex projection algebra — rather than by embedding similarity. In VASM, the query is not a text string but a TERA coordinate specifying an archetypal region of the space: `memory.retrieve(vortexQuery, topK)`. Stored memories are indexed by the TERA coordinate computed at the time of storage (`memory.store(pattern)`). Retrieval returns the top-k memories whose TERA coordinates fall within a specified geodesic radius of the query coordinate on the Vortex-induced (Fisher–Rao) metric. A `memory.probe` operation previews the retrieval without committing to it; `memory.prune(policy)` supports policy-based eviction (e.g., prune all memories in red-shell regions). We argue that TERA-based retrieval is semantically complementary to embedding similarity: embedding similarity

measures surface-form closeness, while TERA proximity measures archetypal-posture closeness. A system may benefit from both, using embedding similarity to retrieve factually relevant content and TERA retrieval to retrieve contextually coherent content. VASM is implemented in MaiiaM Alchemist v0.4.1 as the `memory.*` JSON-RPC namespace.

Keywords: semantic memory, retrieval-augmented generation, archetypal indexing, symbolic retrieval, TERA algebra, Vortex projection, information geometry, Fisher–Rao metric, memory management

---

1. Introduction

> *Series note: This paper is part of the AAMT Working Paper Series. The TERA projection algebra is derived in full in WP-01 (Vortex-Gated MoE). The breath phase system (INHALE/HOLD/EXHALE/EMPTY) and shell classification are derived in WP-03 (PTPS). This paper defines the TERA coordinate space locally (Section 2.1) for self-containedness and directs readers to WP-01 for the complete derivation.*

The dominant paradigm for memory in retrieval-augmented generation (RAG) systems is dense retrieval: a text encoder maps queries and documents to vectors in a shared embedding space, and retrieval is nearest-neighbor search by cosine similarity (Lewis et al., 2020; Karpukhin et al., 2020). This paradigm is powerful for factual retrieval — finding documents that discuss the same topic as the query — but conflates two distinct notions of relevance:

1. **Topical relevance:** does the stored memory discuss the same subject matter as the query?
2. **Contextual coherence:** is the stored memory in the same archetypal posture as the current generation context?

Standard embedding similarity captures topical relevance well. A query about "water cycle evaporation" will retrieve memories discussing meteorology, hydrology, and atmospheric science. But among a large memory store, many topically relevant memories may have been generated in very different archetypal states — one during a high-entropy exploratory generation (INHALE phase, green shell), another during a low-entropy committed generation (HOLD phase, yellow shell). From the model's perspective, the contextually coherent memory is the one generated under similar archetypal conditions, not merely the one with the closest topic.

VASM provides a retrieval axis that standard embedding search cannot: **retrieve what this model was thinking when it was in this archetype**. The TERA coordinate of the query specifies an archetypal region; retrieval returns memories generated in that same region.

---

2. The TERA Coordinate System as a Memory Address Space

2.1 TERA coordinates

A TERA vector $v = (T, E, R, A) \in [0,1]^4$ is a four-dimensional point in the unit hypercube. Every generation context has an associated TERA coordinate, computed by the VortexGate projection of the current

hidden state (WP-01). This coordinate evolves during generation as the model's internal state shifts.

At any point in generation, the current TERA coordinate reflects the model's archetypal posture: `T` (structural coherence), `E` (energetic engagement), `R` (resonant integrity), `A` (active agency). High `A` + high `E` with moderate `T` and `R` indicates an expansive, generative state (green shell). High `T` + high `R` with moderate `E` and `A` indicates a systems-focused, precise state (yellow shell).

2.2 The TERA metric (product Fisher–Rao)

Each Meji distribution is a product of independent Bernoullis (WP-01 §2.2), so the natural information-geometric distance between TERA coordinates factorizes over the four axes and has a closed form. We use the **product Fisher–Rao metric**:

```
ds² = Σ_d  dv_d² / ( v_d (1 − v_d) )                    (per-axis Bernoulli Fisher metric)

d_FR(v₁, v₂) = √( Σ_d ( 2·arcsin√v₁_d − 2·arcsin√v₂_d )² ) (closed-form geodesic distance)
```

Each axis maps to the angular coordinate `θ_d = 2·arcsin√v_d ∈ [0, π]`, in which the Bernoulli Fisher metric is flat (Euclidean); the product manifold is therefore flat in `(θ_T, θ_E, θ_R, θ_A)` and the geodesic distance is the Euclidean norm of the angle differences. The earlier L2 norm `‖v₁ − v₂‖₂` is recovered as the small-displacement limit near `v_d = ½` (where `dθ_d ≈ 2·dv_d`). The KL divergence between Meji measures factorizes correspondingly: `KL(p(v₁)‖p(v₂)) = Σ_d KL_Bernoulli(v₁_d‖v₂_d)`.

Geodesic radius queries — "return all memories within radius r of query TERA q" — are implemented as:

```python
def retrieve_by_tera(query_tera: TeraVector, top_k: int,
                     radius: float = 0.3) -> list[MemoryRecord]:
    candidates = [m for m in memory_store
                  if fisher_rao(m.tera, query_tera) <= radius]
    return sorted(candidates,
                  key=lambda m: fisher_rao(m.tera, query_tera))[:top_k]
```

On the conserved manifold (WP-01 §2.1a) distances are computed within the level set `Σ_κ`. The default radius `0.3` (in Fisher–Rao units) covers a neighborhood corresponding to the same shell and adjacent shells.

---

3. API Design and Semantics

3.1 Implemented interface (v0.4.1)

The `memory.*` namespace is exposed via the Electron preload and routed through the sidecar JSON-RPC:

```
memory: {
    store:    (pattern: MemoryPattern) → MemoryRecord
```

```
    retrieve: (vortexQuery: TeraVector, topK?: number,
               options?: RetrieveOptions) → MemoryRecord[]
    list:     (limit?: number) → MemoryRecord[]
    probe:    (vortexQuery: TeraVector, options?: ProbeOptions) → ProbeResult
    prune:    (policy: PrunePolicy) → PruneResult
}
```

3.2 Storage

`memory.store(pattern)` stores a memory record containing: - The pattern content (text, structured data, or embedding reference) - The TERA coordinate at the time of storage (computed from current sidecar state) - The breath phase at the time of storage (INHALE/HOLD/EXHALE/EMPTY) - A UUIDv7 timestamp - A lineage node ID linking the memory to its generation context

The TERA coordinate is computed automatically from the current engine state — the caller does not need to provide it. This makes storage ergonomic: `memory.store({ text: "..." })` suffices.

3.3 Retrieval

`memory.retrieve(vortexQuery, topK, options)` accepts a TERA coordinate as the query. The caller specifies the archetypal region of interest, not a text string. Options include:

```
{
    radius?: number,          // geodesic (Fisher–Rao) radius (default 0.3)
    shell_filter?: Shell[],   // restrict to specific shells
    phase_filter?: Phase[],   // restrict to memories from specific breath phases
    since?: string,           // ISO-8601 lower bound
    until?: string            // ISO-8601 upper bound
}
```

Shell filtering is particularly powerful: `retrieve(currentTera, 5, { shell_filter: ['green'] })` returns the five memories closest to the current archetypal state that were also generated in green-shell mode. This retrieves contextually coherent high-depth memories regardless of their topical content.

3.4 Probe

`memory.probe(vortexQuery, options)` returns statistics about what a retrieval would find without materializing the memory contents: count of candidates within radius, shell distribution, average Fisher–Rao distance. This is useful for adaptive retrieval strategies: probe first to determine if the memory store has coverage of the query region before committing to a full retrieve.

3.5 Prune

`memory.prune(policy)` supports declarative eviction:

```
type PrunePolicy =
    | { type: 'shell', shells: Shell[] }           // remove all red-shell memories
    | { type: 'age', older_than_days: number }     // remove stale memories
    | { type: 'radius', center: TeraVector, keep_radius: number }  // keep only near center
    | { type: 'count', max_count: number, strategy: 'oldest' | 'farthest' }
```

The `shell` prune policy is particularly relevant for alignment: red-shell memories (generated under low coherence) can be systematically evicted to prevent misaligned context from contaminating future retrievals.

---

4. Complementarity with Embedding Similarity

VASM is not a replacement for embedding-based retrieval — it is a complementary index. The two retrieval axes answer different questions:

	Embedding similarity	TERA retrieval
Query form	Text string	TERA coordinate
Retrieval criterion	Topical closeness	Archetypal closeness
Captures	"About the same thing"	"Generated in the same posture"
Blind to	Generation state	Surface topic
Strength	Factual relevance	Contextual coherence
Failure mode	Topic-adjacent misalignment	Topic-distant coherence

A hybrid retrieval strategy uses both: first retrieve by embedding similarity to find topically relevant memories, then re-rank by Fisher–Rao distance to the current query coordinate to surface the most contextually coherent among them.

```
def hybrid_retrieve(query_text: str, query_tera: TeraVector,
                    top_k: int = 5) -> list[MemoryRecord]:
    # Stage 1: topical candidates via embedding
    embedding_candidates = embed_search(query_text, top_k=20)
    # Stage 2: re-rank by TERA proximity (Fisher–Rao)
    return sorted(embedding_candidates,
                  key=lambda m: fisher_rao(m.tera, query_tera))[:top_k]
```

---

5. Memory Indexing via the Lineage Journal

VASM memories are first-class lineage nodes. Each stored memory writes a `memory.store` node to the lineage journal (WP-04) with:

```
{
  "kind": "memory.store",
  "payload": {
    "content_hash": "sha256:...",
    "tera": [0.72, 0.65, 0.81, 0.44],
    "shell": "green",
    "breath_phase": "EXHALE",
    "pattern_type": "text"
  }
}
```

Retrievals write `memory.retrieve` nodes recording the query coordinate, radius, and result count. This provides a complete audit trail of memory access patterns, enables replay of memory retrieval for debugging, and allows the harvest phase (WP-04) to identify which memories were retrieved most frequently across sessions — a signal for memory importance weighting.

---

6. Comparison with Existing Memory Systems

MemGPT (Packer et al., 2023): Hierarchical memory management with main context and external storage, addressed by recency and importance. No geometric addressing.

Cognitive architectures (ACT-R, Soar): Long-term declarative memory addressed by chunk activation strength, which decays with time and recency. TERA addressing is state-based, not time-based.

Episodic memory for LLMs (Zhong et al., 2024): Key-value stores with summary-based retrieval. Keys are text summaries; retrieval is similarity over summaries. TERA retrieval requires no summary generation.

Atlas (Izacard et al., 2023): Memory as a continuously updated dense retrieval index. VASM's TERA index is independent of the retrieval model's embedding space.

Key distinction: VASM is the first memory system for language models where the address space is defined by a symbolic algebraic projection of the model's internal state — with a principled information-geometric (Fisher–Rao) metric — rather than by embedding similarity, recency, or manually assigned keys.

---

7. Theoretical Properties

Property 1 (Monotone shell coverage): The retrieval radius covers at least one complete shell hyperface of the TERA hypercube for any query in the interior of that shell. Shell-filter retrieval always returns results when the memory store has any entries in the filtered shell.

Property 2 (Probe informativeness): `memory.probe` is strictly faster than `memory.retrieve` for any non-empty result set, because it short-circuits before materializing memory content.

Property 3 (Prune safety): Red-shell prune removes only memories generated during low-coherence states. Green/yellow shell memories are never pruned by a red-shell policy. The policy is monotone with respect to shell ordering: `red < orange < yellow < green`.

---

8. Limitations and Future Work

The current VASM implementation indexes memories by their TERA coordinate at storage time. This is a snapshot of the generation state — it does not capture the trajectory of the TERA coordinate through the generation of the stored content. Future work should explore trajectory-indexed storage: instead of a single TERA point per memory, store the TERA path (the sequence of TERA coordinates during generation, the same "generation worldline" whose speed defines HeartScale drift in WP-02), and query by path similarity.

The information-geometric metric requested as future work in Revision 1 is now adopted in §2.2: for product-Bernoulli Meji measures the KL divergence factorizes and the symmetric Fisher–Rao geodesic has the closed form `d_FR = √Σ_d(2arcsin√v₁_d − 2arcsin√v₂_d)²`. Distances are computed on the conserved manifold `Σ_κ` (WP-01 §2.1a). The remaining open direction is the trajectory indexing above.

---

9. Conclusion

VASM introduces a novel retrieval paradigm for language model memory systems: retrieval by archetypal geometry rather than embedding similarity. The TERA coordinate system provides a four-dimensional address space, equipped with the product Fisher–Rao metric, in which memories are organized by the model's internal state at the time of generation. Retrieval by TERA proximity finds contextually coherent memories; retrieval by shell filter finds alignment-appropriate memories; hybrid retrieval combines topical and contextual relevance. The system is implemented in MaiiaM Alchemist v0.4.1, grounded in the append-only lineage journal for auditability, and is complementary to all existing embedding-based retrieval approaches.

Acknowledgments

The author thanks the AsAManThinks Research community and the MaiiaM development team. Special recognition to Adrienne P. Hancock (AkashMaMa) and Shawanna Whitehead (YahShay NoK) for foundational contributions to the consciousness framework that motivates this work.

Funding

This research was self-funded through AsAManThinks Research. No external funding was received.

Data Availability

The MaiiaM Alchemist v0.4.1 release binary and documentation from which this paper is derived is available at the AAMT research repository. Source code is internal; the specification is fully described herein.

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research, which develops the MaiiaM platform. No competing financial interests exist.

References

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., & Grave, E. (2023). Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(1), 1–43. https://doi.org/10.48550/arXiv.2208.03299

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP 2020* (pp. 6769–6781). https://doi.org/10.18653/v1/2020.emnlp-main.550

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33* (pp. 9459–9474). https://doi.org/10.48550/arXiv.2005.11401

Packer, C., Fang, V., Patil, S. G., Kim, K., Wooders, S., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint*. https://doi.org/10.48550/arXiv.2310.08560

Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of AAAI 2024*. https://doi.org/10.1609/aaai.v38i17.29946

Whitehead, W. (2026). MaiiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*. https://doi.org/10.5281/zenodo.19600795